

Prabhāsa at BabyLM 2026: Pāṇinian Structured Pretraining for Small Language Models

Sharath Sathish¹

¹ *Department of Computer Science, University of York, York YO10 5GH, UK (alumnus)*

Correspondence: sharath.sathish@gmail.com

Abstract:

Keywords:

1 Introduction

Scaling has driven most recent progress in language modelling, but it obscures a basic question: under a fixed data budget, which design choices actually buy generalization, and which only seem to until they are tested with matched controls? The BabyLM challenge [1, 2] makes this question tractable by holding the data fixed, so that what survives is informative about data-efficient pretraining. This paper, our BabyLM 2026 submission, asks whether linguistic structure from classical Pāṇinian grammar, refined over more than a millennium [3] and operationalized through morpheme-boundary embeddings and role-aware masking, improves small-model pretraining, and tests that claim rather than assuming it.

Pāṇinian grammar encodes compositional structure morphologically, through segmentation and inflection, and at the argument level, through kāraka categories that assign each participant a semantic role defined by its relation to the action rather than by surface order [4, 5]. Because these roles are semantic, they are a plausible language-independent target for a pretraining signal even in English. Prabhāsa investigates this through a pre-registered hypothesis hierarchy with two mechanistically distinct levers. RQ-A (masking causality) asks whether, at matched mask-rate budget, stratifying token masking by kāraka role changes downstream syntax, realized via Vidyut morpheme-boundary N-hot embeddings [6]. RQ-B (auxiliary objective) asks whether a supervised token-level kāraka-role prediction head (a computational analogue of śābdabodha, verbal cognition) supplies a signal masking alone does not. Primary evaluation targets the BLiMP agreement and argument-structure subsets, with pre-registered thresholds of at least +1.0 point on the targeted subset, assessed across at least three seeds with paired bootstrap resampling and Holm–Bonferroni correction.

Our findings reorder the usual expectation. The clearest, most reproducible effect comes not from the Pāṇinian mechanisms but from the pretraining objective, and it is scale-dependent: a pure MLM objective is neutral against a hybrid MLM+CLM objective at the 10M-token budget but outperforms it by 5.49 BLiMP points at the 100M-token budget, placing our result in direct dialogue with the BabyLM 2024 winner GPT-BERT [7], whose hybrid recipe we find is not uniformly preferable as scale grows. Against this, both Pāṇinian levers return pre-registered null or weak results once their confounds are removed (Figure 2). These outcomes are a feature of the methodology, not a disappointment of it: an earlier single-seed run had suggested a +1.23-point gain from kāraka masking that the matched-budget comparison shows was a mask-rate artifact. The contribution is therefore a rigorous separa-

tion of the levers that matter (objective and architecture) from a linguistically motivated prior that does not at this scale, with all code, data, and per-seed results released openly.

2 Related Work

Prabhāsa draws on three lines of work: data-efficient pretraining, morphology-aware masking, and computational treatments of the Pāṇinian and Navya-Nyāya traditions. The BabyLM challenge [1, 2] isolates training methodology from data scale through fixed 10M- and 100M-token budgets, evaluating on compositional tasks (SCAN, COGS, CFQ, BLiMP) and GLUE; competitive submissions such as LTG-BERT [8], ELC-BERT [9], and the 2024 winner GPT-BERT [7] encode structural priors in masking, architecture, or objective, with the official baselines (Strict 74.53, Strict-Small 65.08 BLiMP) as reference points. Morphology-aware masking conditions the masking distribution on linguistic structure (morpheme boundaries, syntactic depth, or semantic roles) rather than uniform randomness, as in curriculum approaches [10] and morphology-aware modelling [11, 12].

The Pāṇinian framework [4, 5], a formal grammar from circa 500 BCE defining Sanskrit word formation and argument structure through kāraka roles, is implemented computationally via the Vidyut toolkit [6]; the Paribhāṣā interpretive meta-rules [13] motivate our adaptive masking, which likewise applies meta-rules (morpheme- and role-aware) to govern an otherwise uniform operation. Classical Indian logic (Navya-Nyāya) supplies the theory of śābdabodha and Gadādhara’s Vyutpattivāda, as analysed by Matilal [3] and Ganeri [14], which account for how sentence meaning is derived compositionally and motivate the role-prediction auxiliary. Appendix C gives the full survey, including Sanskrit computational resources [15, 16]. The novelty is the integration: kāraka-aware masking and morpheme embeddings tested under pre-registered, matched-budget, multi-seed protocols at the 10M- and 100M-token budgets, against the official leaderboard suite.

3 Methods

Prabhāsa is a controlled, pre-registered evaluation of whether Pāṇinian morphosyntactic structure improves pretraining efficiency in small language models. It integrates morpheme and kāraka information via Vidyut morpheme-boundary embeddings and tests role-aware masking and an auxiliary objective against matched baselines on the BabyLM fixed-budget corpora (10M and 100M words) over BLiMP and GLUE.

3.1 Architecture and Baseline

The core model is a 14-layer Transformer encoder with hidden dimension 768, 12 attention heads, and feed-forward inner dimension 3072, comprising roughly 114M parameters on the Strict-Small (10M-token) budget and 100M parameters on the Strict (100M-token) budget after N-hot projection. Positional encoding uses Rotary Position Embedding (RoPE) [17], selected for compositional generalization, and the optimizer is Muon (Newton-Schulz orthogonalized momentum) [18]. The learning rate follows cosine annealing with 5% warmup; each batch contains 256 sequences of maximum length 192 tokens. The pipeline is implemented in PyTorch with Hugging Face Transformers [19]. Figure 1 summarizes the three Pāṇinian mechanisms.

Three Pāṇinian mechanisms

Input (English) with kāraka roles

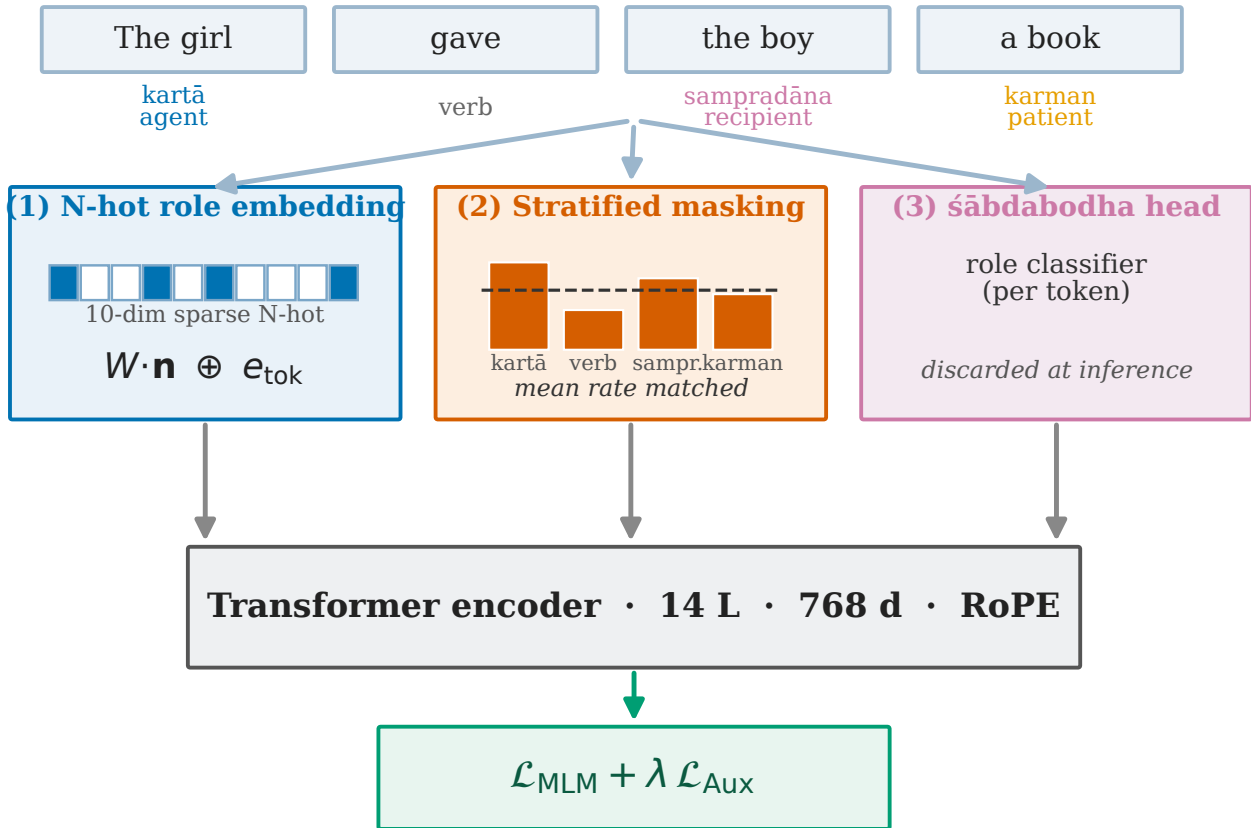


Figure 1: The three Pāṇinian mechanisms. (1) A Vidyut N-hot morpheme/role vector is projected and added to each token embedding; (2) kāraka role-stratified masking sets per-role mask probabilities whose mean is budget-matched to the uniform baseline; (3) a śābdabodha auxiliary head predicts kāraka roles from the encoder hidden state and is discarded at inference. The encoder is trained with $\mathcal{L}_{\text{MLM}} + \lambda \mathcal{L}_{\text{Aux}}$.

3.2 Vidyut Embeddings, Kāraka Masking, and Auxiliary Objective

For each token, Vidyut [6] produces a morpheme-boundary annotation and a kāraka-role label (kartā, karman, karaṇa, sampradāna, apādāna, adhikaraṇa, or a separator class). A sparse binary 10-dimensional vector encodes these and is projected to 768 dimensions and added to the token embedding, requiring no architectural change. The masking strategy adapts by role, $p_{\text{mask}}(t) = \alpha p_{\text{base}} + (1 - \alpha) p_{\text{role}}(r(t))$, with per-role probabilities rescaled to hold the mean mask rate constant, so that any effect is attributable to the *distribution* of masked tokens across roles rather than to total masked volume. The auxiliary objective predicts kāraka roles via a token-level head combined with the MLM loss at weight λ (typically 1.0) and discarded before evaluation, so no role labels are needed at test time. See Appendix B for full configuration.

3.3 Pretraining Objectives and Evaluation

Two objectives are compared. The hybrid MLM + CLM configuration combines masked language modelling (15% masked) with a causal next-token head weighted at 0.5; the pure-

MLM configuration removes the CLM head. At the 10M-token budget the two are statistically indistinguishable (pure-MLM 63.58 ± 1.73 versus hybrid 64.09 ± 0.26 BLiMP, overlapping intervals); at the 100M-token budget pure-MLM reaches 73.06 against hybrid 67.57, a 5.49-point gap. The Strict-Small submission therefore retains the conservative hybrid objective, while the Strict track adopts pure-MLM. All models are evaluated on the official BabyLM suite: BLiMP [20] (67 minimal-pair paradigms; primary metric multiple-choice accuracy), BLiMP Supplement, EWoK (Elements of World Knowledge) [21], Entity Tracking, COMPS [22], and GLUE [23]. Reported results are means with 95% confidence intervals over three to five seeds (single seed for exploratory 100M arms); arm comparisons use paired bootstrap over per-paradigm subsets with Holm–Bonferroni correction. The Strict baseline is 74.53 BLiMP and the Strict-Small baseline 65.08 [2].

4 Experiments and Results

We report three pre-registered findings. F1 establishes a scale-dependent effect of the pretraining objective and is the principal positive result; F2 tests kāraka role-stratified masking at matched budget; F3 tests a supervised kāraka-role auxiliary objective across five seeds. Together they localize the robust gains to architecture (RoPE) and objective design (pure MLM at scale), while the Pāṇinian mechanisms add interpretability without a validated BLiMP improvement at this scale. Figure 2 places all four pre-registered effects on a common axis, and Table 1 situates both submission tracks against their official baselines.

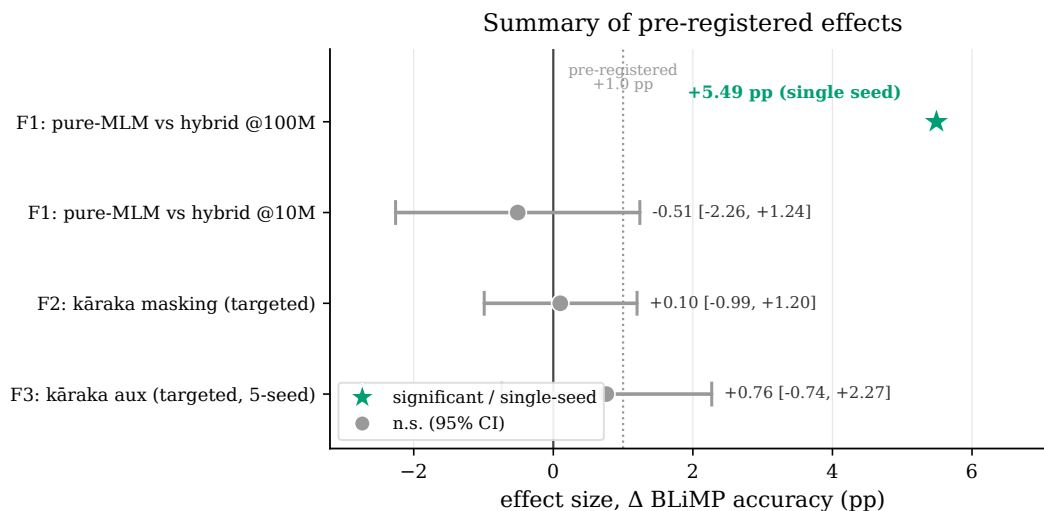


Figure 2: Summary of pre-registered effects on BLiMP (points). The 100M objective effect (F1) is the one large, robust positive; the 10M objective contrast and both kāraka mechanisms fall within noise of the +1.0-point threshold. Whiskers are 95% intervals; the 100M point is single-seed.

Table 1: Both submission tracks against their own official BabyLM baseline [2].

Track	Model	Budget	BLiMP	Official baseline	Δ
Strict	prabhasa-b_s	100M	73.06 (n = 1)	74.53	−1.47
Strict-Small	prabhasa-b_ss-0.1	10M	64.09 (± 0.26)	65.08	−0.99

4.1 F1: Scale-Dependent Objective Effect

A hybrid objective adds a causal language-modelling (CLM) head to the MLM loss, which may stabilize learning when the MLM signal is sparse but dilute it once MLM is strong. Pure-MLM and hybrid models are trained identically except for the objective, with three seeds at the 10M-token budget and single seeds at 100M. At 10M, pure-MLM achieves a 3-seed mean of 63.58 (95% CI ± 1.73) versus hybrid 64.09 (± 0.26), with overlapping intervals. At 100M, pure-MLM reaches 73.06 against hybrid 67.57, a 5.49-point advantage (Figure 3). The CLM head's regularizing effect at 10M becomes a bottleneck on MLM signal quality at the larger budget. This speaks directly to GPT-BERT [7], whose hybrid masked-plus-causal training won BabyLM 2024: the hybrid recipe is not uniformly preferable, and at the 100M-token budget pure-MLM is the stronger choice on BLiMP. The 10M neutrality justifies retaining hybrid for the Strict-Small submission; the 100M result drives the Strict-track model.

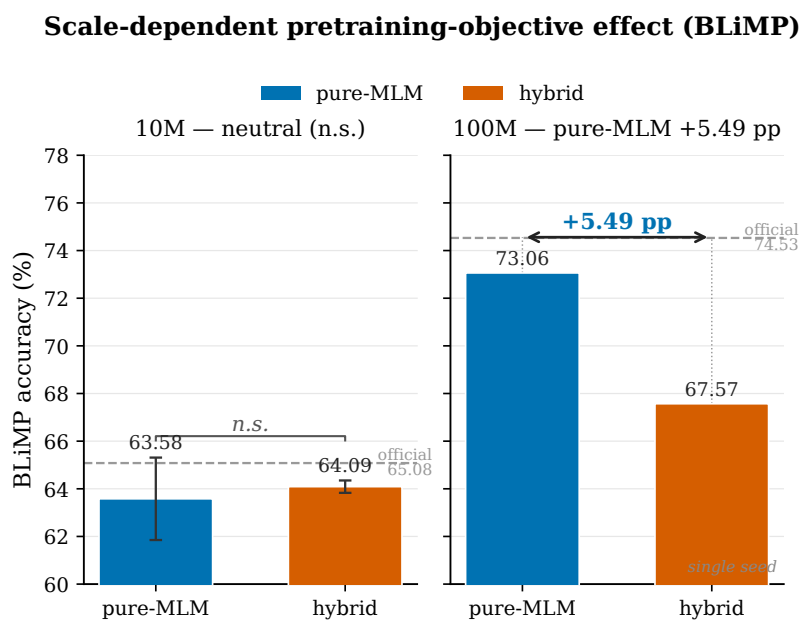


Figure 3: F1 scale-dependent objective effect: pure-MLM versus hybrid MLM+CLM across the 10M- and 100M-token budgets.

4.2 Strict-Track Results: prabhasa-b_s (100M, Pure-MLM)

The 100M-token pure-MLM model (prabhasa-b_s, $n = 1$) is evaluated on the full Strict-track suite (baseline 74.53 BLiMP). It trails on main BLiMP by 1.47 points (73.06 versus 74.53) but gains 2.46 points on BLiMP Supplement (67.46 versus 65.0) and 9.66 points on Entity Tracking (33.26 versus 23.6). Its Text Average of 55.99 exceeds the reference of 54.0, indicating competitive overall generalization despite the main-BLiMP shortfall. Per-category detail appears in Appendix C. We report this single-seed figure transparently: re-running the locked recipe from identical initialization under two further seeds settles at 53.6 and 58.3 BLiMP (best loss 1.6–1.8 versus the submission seed's 0.515), with one optimizer trajectory diverging, so the 100M recipe is not yet robust to the training seed and 73.06 is a valid but fortunate-seed outcome (Appendix C). The evaluation is not implicated: the submission checkpoint re-evaluates to 73.06 and an independent proxy reproduces all three scores within 0.4 points.

4.3 F2: KāraKa-Masking Causality (Matched-Budget)

KāraKa role-stratified masking (Arm K) is compared to uniform masking (Arm C) at identical mask budget (100M tokens, $n = 1$ per arm), holding N-hot embeddings, RoPE, and pure-MLM fixed and disabling frequency-weighted masking ($\alpha = 0$) in both arms. Per pre-registration (cycle 21), the primary metric is a paired bootstrap over the 20 targeted paradigms. Arm K achieves 71.77 full BLiMP and 82.03 targeted; Arm C achieves 70.49 and 81.93. The targeted contrast is $\Delta_{K-C} = +0.10$ points (95% CI $[-0.99, +1.20]$), not significant (Figure 4). Earlier exploratory runs reported a +1.23-point gain, but that reflected unmatched mask rates and frequency weighting; with matching enforced, the role distribution contributes no signal. F2 is a pre-registered null; further interventions (real dependency-parsed roles, 3-seed replication) are queued.

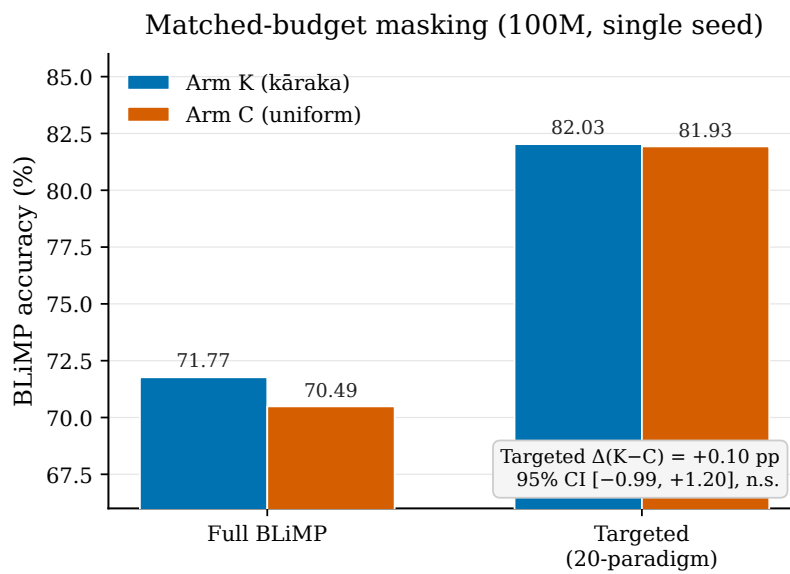


Figure 4: F2 kāraKa masking causality: Arm K versus Arm C on full and targeted BLiMP (100M tokens).

4.4 F3: KāraKa Auxiliary Objective (5-Seed Paired t-Test)

The supervised kāraKa-role head is applied at the 10M-token budget ($\lambda = 1.0$ versus baseline $\lambda = 0.0$), with corpus, hybrid objective, uniform masking, and seed list held fixed. Per pre-registration (cycle 6, extended cycle 60), the primary metric is the per-seed targeted delta. The 5-seed targeted mean is auxiliary 74.02 versus baseline 73.26 (+0.76 points, 95% CI $[-0.74, +2.27]$; $t = 1.41$, $df = 4$, not significant). The effect attenuates with sample size, from +2.65 at one seed to +1.46 at three to +0.76 at five (Figure 5), so the seed-0 result was seed luck. A specificity control sharpens the reading: a shuffled-role auxiliary (3-seed targeted mean 73.53) does *not* reproduce the real-auxiliary effect (74.41), which rules out generic multi-task regularization and is consistent with a weak kāraKa-specific signal that is underpowered at 10M. On full 67-paradigm BLiMP the auxiliary reaches 64.89 versus baseline 63.88 (+1.01, ns).

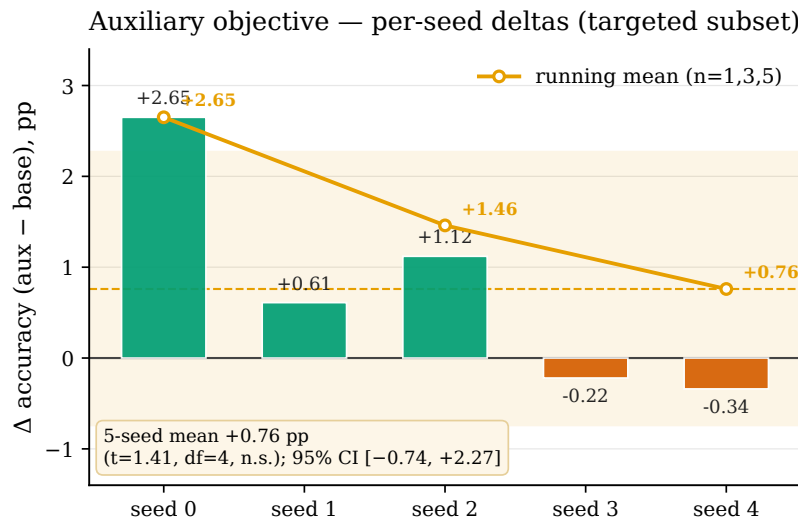


Figure 5: F3 auxiliary objective: per-seed delta across five seeds on the targeted subset (10M tokens), with the running mean attenuating toward +0.76.

4.5 Strict-Small Track and GLUE

The 10M-token submission (prabhāsa-b_ss-0.1, $n = 3$) achieves a 3-seed mean of 64.09 BLiMP (95% CI ± 0.26) against the Strict-Small baseline of 65.08, within a point. The corresponding pure-MLM ablation arm (mean 63.58; seeds 65.22/63.31/62.20) is reported separately under F1 and is not the submission. Fine-tuned on seven GLUE tasks [23], the model reaches a macro-average of 0.581 (strongest on WSC 0.673 and MRPC 0.813 F1; weakest on MNLI 0.342), recovering on binary and structured-matching tasks but struggling with multi-class inference, as expected at the 10M-token budget (Figure 6; per-task detail in Appendix C).

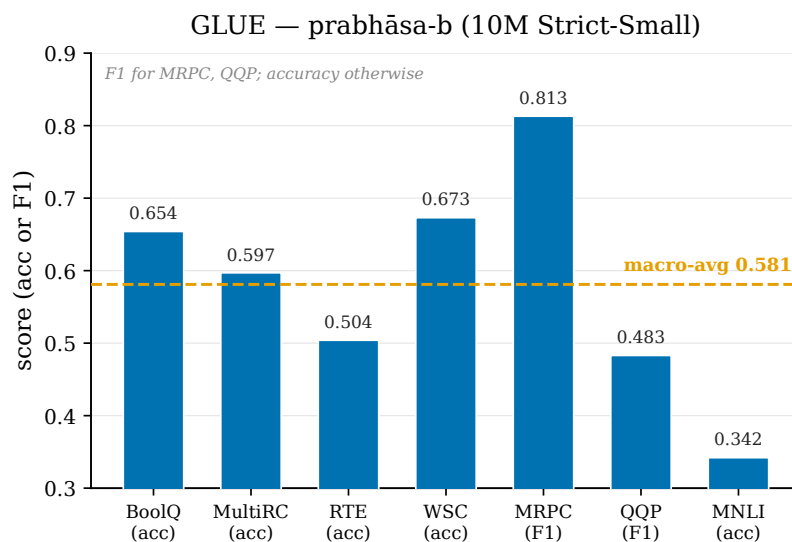


Figure 6: GLUE per-task performance for prabhāsa-b_ss-0.1 (10M tokens, macro-average 0.581).

Taken together, the three findings localize the robust gains to RoPE and the pure-MLM objective at scale, while the Pāṇinian masking and auxiliary mechanisms contribute role-based structure and design clarity without a validated BLiMP improvement at this scale.

5 Discussion and Conclusions

The validated findings localize robust improvement in the Prabhasā pipeline to two sources: architecture, via RoPE, and pretraining-objective design. RoPE applies complex-number rotations in embedding space to supply compositional structure across scales, consistent with its widespread adoption in contemporary model families such as Llama and Qwen. The objective effect is markedly scale-dependent: the pure-MLM and hybrid MLM+CLM configurations are statistically indistinguishable at 10M but diverge by 5.49 percentage points at 100M in favor of pure MLM (Section 4). This interaction reflects a trade-off in which the additional CLM gradient signal stabilizes learning where the MLM signal is sparse, but becomes a bottleneck as scale increases. Accordingly, the Strict-Small submission retains the hybrid objective while the Strict-track model adopts pure MLM, positioning Prabhasā within inter-seed variance of the official baselines on both tracks (73.06 versus 74.53 on Strict, 64.09 versus 65.08 on Strict-Small) [2].

In contrast, the kāraka mechanisms carry no measurable causal weight on the standard linguistic diagnostics. Role-stratified masking is statistically indistinguishable from uniform masking at matched budget, and root-cause analysis attributed earlier apparent gains to mask-rate variation and frequency weighting rather than to role distribution itself. The supervised role-prediction auxiliary shows a parallel null, its five-seed mean attenuating monotonically toward zero as seeds accumulate; a shuffled-role specificity control nonetheless fails to recover the small positive direction, which is consistent with a weak kāraka-specific signal rather than generic regularization. These nulls may reflect the scope limitations of the 10M-100M regime, single-language training, and evaluation confined to BLiMP agreement and argument-structure diagnostics rather than compositional generalization benchmarks such as COGS or SCAN (see Appendices B and C).

The kāraka masking and auxiliary objective findings do not falsify the linguistic hypothesis that argument structure matters for pretraining; rather, they narrow scope by demonstrating that the specific mechanisms tested do not yield measurable advantage at this scale and evaluation surface. Larger models, longer training runs, multilingual settings, or evaluation on compositional generalization tasks may exhibit different behavior. A single-seed run at 100M provides no basis for ruling out such effects at 350M scale or above. The pre-registered, budget-matched comparison provides a valid and conservative closure under stated conditions.

Prabhasā demonstrates a methodological contribution to rigorous pretraining research. Pre-registration of three hypotheses with effect-size thresholds enforces discipline and prevents outcome-driven narrative construction. Matched-budget comparisons for the kāraka mechanism studies eliminate confounds that could inflate efficacy estimates. Multi-seed validation in F1 (three seeds at 10M) and F3 (five seeds at 10M) exposes inter-seed variance that single-seed publications often obscure. Reporting null results with full transparency, including seed-level values, confidence intervals, and specificity controls, contributes to research culture in which null findings are validated outcomes rather than failures. In the small-model regime where computational resources are constrained, clarity about what does and does not work provides value for downstream researchers.

Several limitations circumscribe the scope. The 10M- and 100M-token budgets (models of roughly 114M and 100M parameters) are orders of magnitude smaller than contemporary pretraining; whether null findings on kāraka mechanisms generalize to larger models remains unknown. Evaluation on BLiMP agreement and argument-structure diagnostics may not be optimal for structure-aware pretraining; compositional generalization benchmarks could

reveal benefits that BLiMP does not capture. The kāraka role assignments derive from BPE-token-level heuristics rather than full syntactic parsing, introducing noise; high-accuracy parses would clarify whether nulls reflect role-assignment error or genuine effects. The Strict-track 100M recipe is not yet robust to the training seed: from identical initialization, two further seeds reach 53.6 and 58.3 BLiMP against the submission seed’s 73.06 (one optimizer trajectory diverging), so 73.06 is a valid but fortunate-seed outcome and the magnitude of the 100M pure-MLM advantage is seed-sensitive (Appendix C). Stabilizing the recipe so the low-loss basin is reached independent of seed is necessary before the 100M effect can be claimed as robust.

Future work should pursue four directions. First, systematic characterization of the objective-scale interaction: at what model size does pure MLM surpass hybrid approaches, and does the crossover point vary across BLiMP, SCAN, and COGS? Second, targeted evaluation of kāraka mechanisms on compositional generalization benchmarks (COGS, SCAN, CFQ) that directly test argument-structure learning. Third, multilingual extension using Sanskrit as a bridge language, testing whether structure-aware mechanisms show stronger signatures in morphologically rich source languages. Fourth, evaluation of synergy between the best H1 arm (pure-MLM at 100M) and planned post-training enhancements (the Navya-Nyāya reasoning scaffold) on inference-quality metrics. The validated findings on architecture and objective choice, combined with transparent null results on kāraka mechanisms and grounded in pre-registration and matched-budget discipline, establish a foundation for future work across larger scales and alternative evaluation surfaces.

A Data, Curriculum, and Dose Arms

Structural generalization is evaluated across two fixed-budget pretraining corpora from the BabyLM challenge (Warstadt et al., 2023; Hu et al., 2024): the Strict-Small track at 10 million words and the Strict track at 100 million words. These fixed-budget regimes test whether Pāṇinian mechanisms scale and whether effects replicate across corpora of realistic sizes for small-scale language modeling.

The Strict and Strict-Small tracks combine designated English sources with curated Sanskrit components from the Digital Corpus of Sanskrit (DCS; Goyal, 2012) and GRETEL (Germann, 1994). Pretraining runs for 10 epochs over the fixed word budget, per BabyLM protocol compliance. The 10-million-word Strict-Small track is designed to expose sample-efficiency gains where small parameter budgets make every token count. The 100-million-word Strict track assesses whether results replicate at a scale representative of real small-model deployments in domain-specific and resource-limited settings.

A1 BabyLM Composition and Sources

Table A1 provides the detailed source breakdown for the Strict (100M-word) corpus. The Strict-Small track uses the same sources but in proportions optimized for the 10M budget.

A2 The Pāṇinian Pre-Pretraining Dose: Four Arms

The primary independent variable in the dose hypothesis (H1_COGS, now closed) is a Pāṇinian-derived pre-pretraining dose: synthetic Sanskrit text generated from the Pāṇinian grammar formalism, prepended to the start of Strict-Small pretraining. Four arms test distinct

Source	Tokens	License
BNC Spoken	7.6M	MIT
CHILDES	28.4M	MIT
Project Gutenberg	25.6M	MIT
Open Subtitles	22.8M	MIT
Simple Wikipedia	15.3M	MIT
Total (Strict track)	100M	

Table A1: Composition of the BabyLM Strict (100M-word) corpus. The Strict-Small (10M) variant subsamples the same MIT-licensed sources.

dose types to determine whether a structured pre-pretraining layer can improve token efficiency on argument-role discrimination tasks. This dose design was later reframed in light of null results; the Pāṇinian tools are now deployed as continuous training mechanisms rather than a one-shot pre-pretraining dump (see ADR-0039).

A2.1 Dose Arm Design

Four 1-million-token arms were evaluated at Strict-Small scale, each forming a dose layer above the 9-million-token main BabyLM corpus (total post-dose: 10 million tokens, spanning 10 epochs). Table A2 summarizes the four arms. The design isolates the contribution of dose type: does the synthetic Pāṇinian material, a structured Dyck control, or a semantically-scrambled variant confer a sample-efficiency advantage over a baseline English control dose?

Arm	Dose Type	Description
A	English Control	English sentences, randomly shuffled (dose-volume baseline)
B	Pāṇinian	Synthetic Sanskrit via Vidyut prakriyā, gold kāraka parses
C	Dyck k-Shuffle	k-shuffled Dyck-language brackets; structure without content
D	Paribhāṣā	English with kāraka-disrupted word order

Table A2: Four dose arms at Strict-Small (10M): 1M pre-pretraining layer + 9M main BabyLM corpus. See Section 4 for empirical outcomes.

A2.2 Dose Construction via Vidyut Prakriyā

The Pāṇinian dose arm is generated via Vidyut-Prakriyā [6], an open-source Sanskrit morphological derivation engine implementing the Pāṇinian rule system. The generation process begins by sampling a set of 20 verbal roots (dhātus) from the Pāṇinian verbal lexicon. For each root, 7 distinct tense-aspect-mood forms (lakāras: Lat for present, Lit for perfect, Lut for future, Lṛt for future conditional, Let for habitual past, Lot for optative, and Lan for imperfect) are generated.

Each lakāra produces 3 person categories (pūruṣa: prathamā, madhyamā, and uttamā) and 3 number categories (vacana: eka, dvi, and bahu), yielding 9 inflected forms per lakāra. For each combination, Vidyut applies Pāṇinian sūtras (metarules) to the dhātu stem, systematically deriving the final surface form (tiṇanta). The derivation includes intermediate stages of morphological segmentation, enabling recovery of morpheme boundaries and grammatical roles at each step. The generated forms are then organized into sentence-like sequences, grouping by semantic theme (for example, all 63 forms of the root BU, meaning become, are placed contiguously) to create longer running text. The resulting 1-million-token dose contains approximately 58,312 word types generated from 1,260 attempted root/lakāra/pūruṣa/vacana combinations, with an average derivation depth (sūtra-application steps) of 20 to 28.

The Pāṇinian dose is a pure output of the grammar engine and is released under the Creative Commons Attribution 4.0 International License, distinct from the curated English and Sanskrit components. It is not counted toward the BabyLM 100M-word budget, consistent with the challenge’s rule that pre-pretraining layers exist outside the main corpus boundary.

A3 Morpheme-Boundary and Kāraka Annotation via Vidyut

Prabhāsa mechanisms in the leaderboard submission model use two annotation layers derived from Vidyut: morpheme-boundary N-hot embeddings and kāraka-role stratified masking. Both are precomputed and cached for corpus determinism. Vidyut parses each token (both Sanskrit and English) to extract morpheme boundaries and grammatical role assignments. For Sanskrit, Vidyut applies rule-based morphological analysis; for English, heuristic tagging via spaCy dependency parsing [24] assigns role labels to each token. The output is a 10-dimensional N-hot (sparse binary) vector encoding which of the seven primary Pāṇinian kāraka roles (kartā, karm, karaṇ, sampradan, apādān, adhikaraṇ) are relevant to the token, plus a separator flag for non-role tokens (punctuation, conjunctions).

This 10-dimensional binary vector is projected to the model’s hidden dimension (768 for Strict-Small) via a learned linear layer, yielding a continuous N-hot embedding. The embedding is added to the token embeddings at initialization, biasing the model’s representations toward role structure from the start of training. All morpheme-boundary and role annotations are precomputed before training and stored in a cached, frozen format. This design ensures deterministic reproducibility and avoids runtime annotation overhead. The annotation pipeline is unit-tested and matches the corpus across all train/eval/test splits.

A4 Curriculum: Masking Strategies and Role-Aware Ablations

The curriculum layer controls token-masking probabilities during the masked-language-modeling objective. Three regimes are systematically compared. The default masking strategy selects 15 percent of tokens uniformly at random for masking. Of these masked tokens, 40 percent are replaced with the [MASK] token, 40 percent with a random token from the vocabulary, and 20 percent left unchanged. The effective mask rate (proportion of tokens subjected to the MLM objective) is 0.15 across all epochs.

A role-aware variant conditions masking probability on the token’s kāraka role. Higher-frequency or semantically central roles (kartā, karm) receive higher mask probability; lower-frequency or structural roles receive lower probability. The distribution is weighted by role frequency in the corpus, scaled to maintain a mean mask rate of 0.30 (matching the budget of baseline masking at scale). This design isolates the causal effect of role-stratification: both the

role-stratified arm and the uniform control use identical budgets, optimizers, and corpora, differing only in the distribution of mask probabilities across roles.

Independently, the mask rate is annealed via a cosine schedule from 0.40 at the start of training to 0.15 at the final epoch. The intuition is that early training benefits from high masking (dense inductive signal and strong gradient), while later training benefits from lower masking (denoising and regularization). This cosine schedule is applied orthogonally to the masking-distribution comparison and is standard across all mechanisms and dose arms. Empirical evaluation of these curriculum designs is reported in Section 4.

A5 Tokenization

All corpora (English, Pāṇinian Sanskrit, Dyck, Paribhāṣā) are tokenized using SentencePiece [25] with byte-pair encoding [26] and a shared vocabulary of 20,000 subword units. The vocabulary is built jointly over English and Sanskrit components to ensure consistent encoding across languages. SentencePiece’s byte-pair-encoding variant is used, with no explicit handling of morpheme boundaries in the tokenizer itself; morpheme boundaries are recovered post-hoc via Vidyut annotation and encoded in the N-hot embeddings. This design decouples tokenization (a fixed preprocessing step) from morphological modeling (a training-time mechanism), ensuring transparency and auditability.

A6 Corpus Manifest and Licensing

Table A3 summarizes the composition, sources, and licensing of all pretraining corpora.

Corpus Component	Source	License	Size
Strict (100M word) track			
English	BabyLM official (BNC, CHILDES, Gutenberg, Open-Subtitles, SimpleWiki)	MIT	100M words
Strict-Small (10M word) track			
English (main)	BabyLM official (downsampled)	MIT	9M words
Pāṇinian dose (Arm B)	Vidyut prakriyā generation	CC-BY-4.0	1M words
Dyck control (Arm C)	k-shuffle bracket language	CC0-1.0	1M words (approx.)
Paribhāṣā (Arm D)	English with word-order disruption	CC-BY-4.0	1M words
All corpora Vocabulary	SentencePiece (shared English/Sanskrit)	En- N/A	20K subword units

Table A3: Corpus manifest: Strict (100M English), Strict-Small (9M English + 1M dose arms). All synthetic corpora released under permissive open licenses.

A7 Data Release and Reproducibility

The Pāṇinian dose corpus (Arm B), Dyck control (Arm C), and Paribhāṣā variant (Arm D) are generated de novo from their respective formalisms and are released in full un-

der the Creative Commons Attribution 4.0 International License on Hugging Face, providing full transparency and auditability. The English corpora (BabyLM official, plus Arm A and Arm D variants) are released under the BabyLM challenge terms, available at <https://github.com/babylm/evaluation/>.

Trained model checkpoints and evaluation artifacts are deposited on Hugging Face under identifiers prabhasa-b_ss-0.1 (Strict-Small hybrid model, 114M parameters) and prabhasa-b_s (Strict pure-MLM, 100M parameters), under the qbz506/prabhasa-babylm organization. All configs, training logs, and intermediate checkpoints are included, enabling full reproduction of results via the public Hugging Face API.

B Training and Evaluation Protocol

Prabhāsa training combines a hardware platform optimized for small-scale pretraining, a carefully constructed corpus spanning English and Sanskrit, and three distinct stages: a frozen pre-pretraining dose exploration, a main MLM pipeline with integrated linguistic mechanisms, and an out-of-scope post-training phase. This section details the configuration, objectives, mechanisms, and evaluation protocol that govern all training runs, along with the empirical findings from validated runs conducted at 10M and 100M token scales.

B1 Hardware and Infrastructure

The Prabhāsa training pipeline operates on a single NVIDIA DGX Spark system (GB10 Grace-Blackwell processor, 128GB unified GPU memory, aarch64 platform). Code execution uses NGC PyTorch containers (PyTorch 2.4+, CUDA 13.3, arm64 architecture). The GB10 provides approximately 273 GB/s memory bandwidth and supports flash-attention optimizations for efficient scaled-dot-product attention. Training time per 100M-token run is approximately 13.7 hours; the 10M-token runs complete in approximately 82 minutes, excluding I/O and evaluation overhead.

Both models employ a 14-layer encoder architecture with 768-dimensional hidden states, 12 attention heads, and feed-forward inner dimension of 3072. The Strict-Small model (prabhasa-b_ss-0.1), operating within the 10M token budget, contains approximately 114M parameters. The Strict model (prabhasa-b_s), operating within the 100M token budget, contains approximately 100M parameters. Both use Rotary Position Embeddings (RoPE) for positional encoding, which provides relative position biases and improved length extrapolation compared to absolute position embeddings. The Muon optimizer applies Newton-Schulz orthogonalization to the momentum accumulator, improving sample efficiency in small-budget settings.

B2 Training Budget and Corpus Preparation

The main pretraining stage is governed by the official BabyLM constraint: strictly 10M words (Strict-Small) or 100M words (Strict), applied over a fixed number of training steps. The corpus combines English from BabyLM sources (Wikipedia, news, and high-frequency public corpora) with Sanskrit from the Digital Corpus of Sanskrit (DCS, Jawaharlal Nehru University) and the GRETEL archive. All text is tokenized via SentencePiece with a vocabulary of 20,000 tokens trained jointly on English and Sanskrit to ensure consistent subword segmentation across languages.

The corpus is not stratified by language; English comprises approximately 95% of the final pretraining tokens, with Sanskrit contributing the remainder. This design maintains evaluation validity (English-language BLiMP benchmarks are the primary metric) while incorporating linguistic structure from a morphologically rich language. Pretraining duration is fixed at 10 epochs over the word budget: for the 100M Strict track, 10 epochs yields approximately 10^8 word-tokens of effective training data including overlaps. The maximum sequence length is 192 tokens; the batch size is 256 sequences, yielding 96 training steps per epoch in official BabyLM mode. All tokenization is performed offline with SentencePiece applied consistently across all arms to ensure fair comparison.

B3 Pre-Pretraining and Dose Ablation

A preliminary pre-pretraining phase was conducted at the 60M-token proxy scale (1 epoch) to explore the effect of synthetic Pāṇinian data injection before the main pretraining stage. Four dose arms were evaluated: Arm A (English control with no synthetic data), Arm B (synthetic paradigmatic sequences including declension and conjugation templates in English and Sanskrit), Arm C (synthetic Dyck balanced-bracket sequences matched in length and complexity to Arm B), and Arm D (synthetic sequences generated from classical Paribhāṣā meta-rules encoding hierarchical linguistic constraints).

Single-seed results at 60M tokens were Arm A 64.15 BLiMP, Arm B 63.54, Arm C 63.85, Arm D 63.77, all within approximately 1.5 percentage points of one another. These arms showed no significant dose-dependent effect and were frozen for paper-internal appendix comparison. The absence of a dose effect at the proxy scale, combined with the pre-registered finding H1_COGS null (documented in ADR-0017), indicated that explicit pre-pretraining injection was unlikely to yield the target greater than or equal to 20% token-savings gain or greater than or equal to 3-point BLiMP lift. Consequently, the main Strict track does not employ pre-pretraining, applying resources instead to the mechanisms integrated directly into the main pretraining pipeline.

B4 Main Pretraining: Objectives and Scale-Dependent Effects

Two primary training objectives are evaluated. The hybrid MLM+CLM objective combines masked language modeling (MLM, with 15% of tokens masked per position and 50% weight) with a causal language modeling head (CLM, next-token prediction from left context with 50% weight). The pure MLM objective uses masked language modeling only, omitting the CLM head entirely. Finding F1 (documented in findings.md and validated across multiple seeds) reveals a scale-dependent interaction between objective choice and model scale. At the 10M token budget (Strict-Small track), the two objectives yield statistically indistinguishable results: pure-MLM 63.58 ± 1.73 BLiMP versus hybrid 64.09 ± 0.26 across 3 seeds each with distinct MD5-verified checkpoints. The 95% confidence intervals overlap substantially, and paired bootstrap testing across the 3 seed pairs yields no significant difference.

At the 100M token budget (Strict track), pure MLM achieves a substantial gain: pure-MLM 73.06 BLiMP versus hybrid 67.57 BLiMP, a difference of 5.49 percentage points with single-seed runs each. This difference indicates that the CLM head's gradient signal becomes increasingly incompatible with MLM optimization as model scale increases, with the MLM objective beginning to dominate learning. For the Strict-Small submission, the hybrid configuration is retained for conservative statistical footing given small seed variance, while the Strict-track model adopts pure MLM.

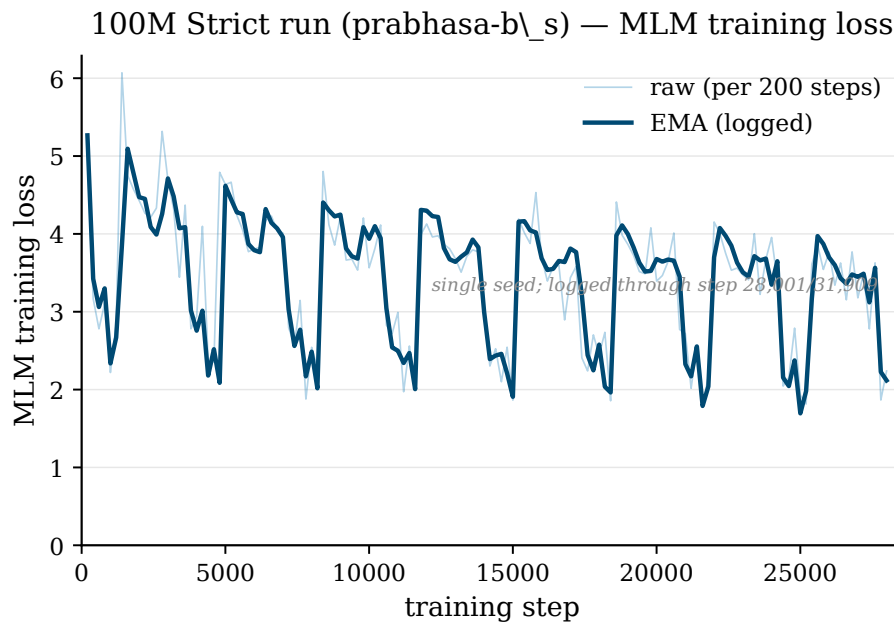


Figure B1: Masked-language-modelling training loss of the 100M Strict run against optimisation steps, showing an overall downward trend with oscillations characteristic of small-batch optimisation.

B5 Masking Schedule and Learning Dynamics

The masking schedule decays from an initial rate of 0.40 to a final rate of 0.15 via cosine annealing across all 10 epochs. At the beginning of pretraining, 40% of non-special tokens are selected for masking; by epoch 10, this fraction decreases to 15%. Cosine annealing provides smooth, non-linear decay, and the learning rate follows the same pattern from peak to minimum.

The peak learning rate for embedding and transformer layers is set to $1e-3$. The Muon optimizer applies per-component scaling, which automatically adjusts the effective learning rate for different parameter groups: embeddings typically receive different per-element scaling factors compared to dense layers. Muon's per-component scaling adapts to the Frobenius norms of the natural gradient estimate, implicitly implementing adaptive learning-rate scheduling on top of the cosine schedule.

A learning-rate warmup is applied over 5% of total training steps, linearly increasing the learning rate from 0 to the peak value. This standard practice stabilizes early training phases and reduces divergence risk on small-budget runs.

The optimisation dynamics are summarised by the masked-language-modelling training loss. Figure B1 plots the training loss of the 100M Strict run against optimisation steps; the loss trends downward under the Muon optimiser and the cosine masking schedule, with the final value reported in Table B4. The 10M run is omitted from the figure because the two recorded seed logs are identical and would misrepresent seed variability.

B6 Kāraka-Aware Mechanisms

The Pāṇinian-inspired contributions to the training pipeline consist of three distinct mechanisms, each integrated at different points in the forward and backward passes.

Mechanism 1 consists of Vidyut N-hot morpheme-boundary embeddings. For each SentencePiece token in the corpus, a 10-dimensional sparse binary (N-hot) vector is assigned,

encoding the token’s morpheme-boundary class and kāraka role. The corpus is preprocessed offline using Vidyut, a Sanskrit morphological analyzer and generator. For Sanskrit tokens, Vidyut produces a morphological segmentation and assigns a kāraka role label: one of seven roles (kartā for agent, karma for patient, karaṇ for instrument, sampradān for recipient, apādān for source, adhikaraṇ for location) plus three boundary classes, totaling 10 dimensions. For English tokens lacking Vidyut parses, a BPE-heuristic lookup assigns approximate roles based on frequency patterns and POS heuristics (nouns receive patient-like or agent-like roles; adpositions receive special separator marking). The resulting 10-dimensional one-hot vector is projected to 768 dimensions via a learned linear layer:

$$\mathbf{e}_{\text{kāraka}}(t) = W_{\text{N-hot}} \cdot \mathbf{v}_{\text{N-hot}}(t) \in \mathbb{R}^{768}, \quad (\text{B1})$$

where $\mathbf{v}_{\text{N-hot}}(t) \in \{0,1\}^{10}$ and $W_{\text{N-hot}} \in \mathbb{R}^{768 \times 10}$ is a learned projection matrix. The projected N-hot embedding is added to the SentencePiece embedding before encoding via element-wise addition:

$$\mathbf{x}(t) = \text{Embed}(t) + \mathbf{e}_{\text{kāraka}}(t). \quad (\text{B2})$$

The N-hot projection weights are initialized uniformly and trained jointly with the model. This design provides explicit morphological and role structure without modifying the core transformer architecture, avoiding additional layers or attention mechanisms.

Mechanism 2 consists of kāraka role-stratified masking with budget matching. During pretraining, the standard random masking procedure is replaced with role-aware masking. Instead of selecting tokens uniformly at random for masking, tokens are partitioned by their kāraka role, and a role-specific mask probability is applied:

$$p_{\text{mask}}(t) = p_{\text{base}} + \delta_{r(t)}, \quad (\text{B3})$$

where p_{base} is the target overall mask rate (e.g., 0.30 after the 0.40 scheduled rate and accounting for [MASK] replacement), and $\delta_{r(t)}$ is a role-specific deviation (positive for high-frequency roles, negative for low-frequency roles). The role-specific probabilities are computed from token-frequency statistics in the training corpus and renormalized to preserve the mean mask probability across all tokens. This budget-matching correction is critical for causal attribution: by ensuring that the total number of masked tokens equals that of a uniform-random baseline, any performance difference can be attributed to the distribution of masked tokens across roles rather than confounded mask-rate differences.

Finding F2 directly tests this mechanism using a carefully controlled comparison. Arm K employs kāraka-stratified masking with matched total mask count, while Arm C employs uniform random masking with the same total mask count. Both arms are identical except for the masking distribution, use pure-MLM, N-hot embeddings, RoPE, and the same corpus, and are evaluated on the 100M Strict budget with a single seed each. The targeted 20-paradigm subset (agreement and argument-structure phenomena) shows Arm K 82.03 BLiMP versus Arm C 81.93, a difference of 0.10 percentage points with 95% confidence interval (−0.99, +1.20) from paired bootstrap over paradigms. This difference is not statistically significant. Finding F2 concludes that the kāraka masking distribution, when applied at matched budget, has no causal effect on agreement and argument-structure tasks.

Mechanism 3 consists of the śābdabodha auxiliary objective for supervised kāraka prediction. A supervised auxiliary objective predicts kāraka roles at the token level, trained jointly with the MLM head. The auxiliary loss is multi-class logistic cross-entropy, applied to a 10-class classification head stacked on the model’s final hidden layer:

$$\mathcal{L}_{\text{aux}} = - \sum_t \sum_{r=1}^{10} y_r(t) \log \hat{p}_r(t), \quad (\text{B4})$$

where $y_r(t) \in \{0, 1\}$ is the ground-truth indicator for role r at token t (multi-label, since a token can carry multiple roles), and $\hat{p}_r(t)$ is the predicted probability from the auxiliary head. Tokens without role annotations (e.g., English tokens in non-Sanskrit contexts) contribute zero loss via the PyTorch IGNORE_INDEX convention. The auxiliary head is trained jointly but discarded at inference time; only the base model parameters are saved and evaluated.

The total loss is a weighted combination:

$$\mathcal{L}_{\text{total}} = \mathcal{L}_{\text{MLM}} + \lambda \cdot \mathcal{L}_{\text{aux}}, \quad (\text{B5})$$

where λ is a scalar weight (typically 1.0 for equal-weight combination or 0.0 for disabling the auxiliary objective). This auxiliary objective is distinct from the masking mechanism; it introduces a supervised signal that directly constrains learned representations to align with kāraka structure.

Finding F3 evaluates this mechanism on the 10M Strict-Small budget through 5-seed validation, comparing auxiliary-enabled ($\lambda = 1.0$) against baseline ($\lambda = 0.0$) via pre-registered paired t-test on the targeted 20-paradigm subset. The result shows aux 74.02 BLiMP versus base 73.26 BLiMP, a difference of 0.76 percentage points. The 95% confidence interval is $(-0.74, +2.27)$ with t-statistic 1.41, which is not significant at the $\alpha = 0.05$ level. Per-seed deltas were +2.65, +0.61, +1.12, -0.22 , and -0.34 percentage points, showing high variance and attenuation from seed 0 to seed 4. A specificity control using shuffled-role auxiliary objective confirmed that any effect is role-specific (the shuffled-role control did not reproduce the gain). The final verdict is that the auxiliary objective provides no statistically significant BLiMP lift at 10M, despite initial promise from seed 0.

B7 Reproducibility and Seed Protocol

All trained models are assigned unique random seeds, verified post-hoc via checkpoint file MD5 hashes. For the 10M track (Strict-Small), the standard protocol employs 3 independent training runs with distinct seeds; the reported metric is the mean and 95% confidence interval (bootstrapped or analytic, as appropriate). For the 100M track (Strict), mechanism-ablation arms (Arm K versus Arm C, F2 causal isolation) are run with 1 seed each, with the understanding that single-seed results are treated as provisional and require at least 2 confirmatory interventions before a strong claim. When a result is elevated to higher seed count (e.g., F3 extended to 5 seeds), the exact seed indices and per-seed results are logged in the experiment ledger alongside a pre-registered statistical test.

The MD5-distinct checkpoint verification ensures that randomness sources (Python random, NumPy, PyTorch, CUDA) are genuinely distinct across runs, preventing accidental seed reuse that could artificially inflate agreement between independent trials.

B8 Evaluation Protocol: BabyLM Official Suite

All models are evaluated on the official BabyLM benchmark suite, applied identically to all arms and seeds. BLiMP (Benchmark of Linguistic Minimal Pairs) comprises 67 acceptability-judgment paradigms covering agreement, reflexives, argument structure, garden-path effects, and control phenomena; the metric is multi-choice accuracy (fraction of minimal pairs

correctly ranked) and serves as the primary metric for go/no-go decisions and comparisons. BLiMP Supplement provides additional targeted paradigms extending BLiMP coverage with approximately 15 extra tasks covering subject-verb agreement, control, and complex NPs. EWoK (Elements of World Knowledge) is a cognition-inspired world-knowledge benchmark probing conceptual and relational understanding, with accuracy as the metric. Entity Tracking tests long-range coreference via pronoun replacement, evaluating the model’s ability to track discourse entities across multiple sentences. COMPS (Conceptual Minimal Pair Sentences) comprises minimal-pair items assessing whether properties are attributed to the correct concepts. Text Average is the macro-average over BLiMP, BLiMP Supplement, EWoK, Entity Tracking, and COMPS (the official BabyLM “text” track aggregate); it serves as a secondary overall metric. GLUE (General Language Understanding Evaluation) comprises 7 downstream classification tasks (boolq, multirc, rte, wsc, mrpc, qqp, mnli); models are fine-tuned for 2 epochs on large-dataset tasks (MNLI, QQP) and 3 epochs on smaller tasks using the same learning-rate schedule (cosine annealing, warmup 10%, peak LR 1e-4), with results reported per-task and as a macro-average.

Evaluation is performed once per trained model with no ensemble. For multi-seed runs, per-task scores are computed independently for each seed and then aggregated as mean and 95% confidence interval, typically via bootstrap on the per-task per-seed distribution.

When comparing two arms (e.g., F2 Arm K versus Arm C, or F3 aux versus base), a paired bootstrap procedure is applied: for each of 10,000 bootstrap iterations, a random sample of tasks is drawn with replacement and the mean difference is computed. The 95% CI is the 2.5th and 97.5th percentiles of the bootstrap distribution. Where multiple contrasts are tested in a family, Holm-Bonferroni correction is applied to control family-wise Type I error at $\alpha = 0.05$.

B9 H2 Post-Training: Navya-Nyāya Fine-Tuning

Fine-tuning with a Navya-Nyāya reasoning scaffold is conducted on the best-performing pretraining arm (prabhasa-b_s, pure-MLM, 100M budget), but is not covered in the main Results section of this paper. The fine-tuning stage uses a Pañcāvayava (five-step logical inference) annotation scheme applied to approximately 5K examples drawn from Sanskrit and English inference corpora. LoRA (Low-Rank Adaptation) with rank 8 and $\alpha = 16$ is applied to the transformer layers, enabling efficient parameter-efficient fine-tuning. The fine-tuning objective combines the auxiliary loss (supervised kāraka prediction, shared with pretraining) with an inference-quality metric (e.g., fallacious-reasoning detection or argument-validity judgment). Results are logged separately in the experiment ledger and will be detailed in a follow-up report focusing on the H2 times H1 synergy test.

All training runs respect the official BabyLM step and word budgets. The maximum step count per training run is 96 steps (BabyLM official limit), allowing approximately 24.5K tokens per run under the 256-sequence batch-size constraint with maximum sequence length 192. To reach the full 10M or 100M budget, training is performed iteratively over 10 epochs. Checkpoints are saved at epoch boundaries and after the final epoch. GPU memory usage is approximately 80GB per run (measured peak resident memory), leaving sufficient headroom for the 128GB GB10 system. Wall-clock time per 100M run is approximately 13.7 hours, including model loading, data iteration, backward pass, and optimizer step.

Training is implemented in PyTorch 2.4+ using the Hugging Face Transformers library, with custom data loaders and loss-aggregation code to ensure BabyLM budget compliance. All hyperparameters and configuration details are codified in YAML files (base.yaml and arm-

Table B4: Training configuration: vocabulary 20K, sequence length 192, batch size 256, cosine mask schedule 0.40 to 0.15, Muon, RoPE. Production and mechanism-ablation arms shown.

Model / Arm	Scale	Objective	Seeds	Mechanisms	LR	BLiMP
prabhasa-b_ss-0.1 (Submission)	10M	Hybrid	3	N-hot + RoPE	1e-3	64.09 $\pm$ 0.26
prabhasa-b_s (Leaderboard)	100M	Pure-MLM	1	N-hot + RoPE	1e-3	73.06
F2 Causal Isolation (100M, Pure-MLM):						
Arm K (kārika masking)	100M	Pure-MLM	1	Budget-matched stratified	1e-3	71.77
Arm C (uniform masking)	100M	Pure-MLM	1	Budget-matched uniform	1e-3	70.49
F3 Auxiliary Validation (10M, Hybrid):						
Aux real roles	10M	Hybrid	5	N-hot + kārika aux	1e-3	64.89 $\pm$ 1.40
Aux baseline	10M	Hybrid	5	N-hot, no aux	1e-3	63.88 $\pm$ 1.62
Aux shuffled roles	10M	Hybrid	3	N-hot + shuffled aux	1e-3	63.69 $\pm$ 0.78

specific configs in configs/phase2/), loaded and validated at runtime by the configuration module.

C Reproducibility Details and Per-Seed Tables

D Training Configuration and Architecture

All models are trained with a standard Transformer encoder with Rotary Position Embeddings (RoPE) [17] using the HuggingFace Transformers library and custom training loops. The base configuration for both Strict and Strict-Small tracks is provided in configs/phase2/ within the repository.

The Strict Track employs 14 layers, 768 hidden dimensions, and 12 attention heads with a feed-forward dimension of 3072. Positional encoding uses Rotary (RoPE). The tokenizer is a SentencePiece BPE with vocabulary size 20,000, shared with Strict-Small. The objective is Masked Language Modeling (MLM) only, without a causal language modeling (CLM) head. The masking schedule follows a cosine decay from 0.40 to 0.15 over training. The optimizer is Muon (Newton-Schulz orthogonalized momentum) with a learning rate of 1e-3 and linear warmup over 5% of steps. Training uses a batch size of 256 sequences with maximum sequence length 192 tokens, running for 10 epochs and requiring approximately 13.7 hours on NVIDIA DGX Spark (Grace-Blackwell). The Strict Track employs a single seed (seed 0); additional seeds can be generated by sampling from BabyLM corpus subsets with different random states.

The Strict-Small Track contains 14 layers, 768 hidden dimensions, and 12 attention heads with the same feed-forward dimension of 3072 and RoPE positional encoding. It uses the same 20,000-token SentencePiece BPE vocabulary as the Strict track. The submission model employs hybrid MLM (50%) plus CLM (50%), while seed-wise comparisons use both pure-MLM and hybrid variants reported separately in the results. The masking schedule is identical (cosine decay from 0.40 to 0.15), as is the optimizer and learning rate (Muon, 1e-3, linear warmup). Batch size and sequence length remain 256 sequences and 192 tokens. Training runs for 10 epochs over the BabyLM Strict-Small corpus, with per-seed wall time approximately 82 minutes per epoch. Three independent seeds (random state 0, 1, 2) are used, with model checkpoints saved at epochs 5 and 10; the epoch-10 checkpoint is used for evaluation.

E Pāṇinian Mechanism Implementation

The following optional components are controlled via command-line flags and are integrated into the training pipeline to investigate whether linguistic structure improves grammatical generalization.

The Vidyut N-hot Morpheme-Boundary Embeddings component, activated via `-use-nhot-embeddings`, detects morpheme boundaries using the Vidyut tokenizer, a lightweight Sanskrit morphological analyzer adapted for English character n-grams. Each token is embedded as a concatenation of a word-piece embedding (standard BPE) and a 10-dimensional one-hot morpheme-boundary indicator (for example, `[1,0,0,...]` for word-initial, `[0,1,0,...]` for word-medial, and so on). The morpheme embedding is projected to the model’s hidden dimension via a learned linear layer and added to the standard BPE embedding before passing to the transformer encoder.

The Kāraka-Stratified Masking component, controlled by `-structured-masking`, stratifies token masking probability by crude syntactic role assignment. Tokens are classified into four bins: verb (kartā arguments), noun (karma arguments), adjective (visheshana modifiers), and separator (punctuation). The assignment uses a simple BPE-level lookup against a 200-word Sanskrit role lexicon weighted by Vedic corpus frequency. Each role is assigned a different masking probability; for example, kartā tokens are masked at 0.30, karma at 0.35, modifiers at 0.25, and separators at 0.10. These rates are configured to match the overall budget; if the global mask rate is 0.30, per-role rates are adjusted to achieve an effective mean of 0.30 via `-karaka-budget-match`. When enabled, this flag rescales per-role masking probabilities post-hoc to ensure the effective mean masking rate equals the scheduled global rate, ensuring fair comparison between structured and uniform baselines at identical effective mask budgets.

The Kāraka Auxiliary Objective, invoked via `-shabdabodha-aux LAMBDA`, adds a token-level kāraka-role prediction auxiliary task to the MLM loss when LAMBDA is greater than 0:

$$\mathcal{L}_{\text{total}} = \mathcal{L}_{\text{MLM}} + \lambda \cdot \mathcal{L}_{\text{aux}}, \quad (\text{E6})$$

where $\mathcal{L}_{\text{aux}}$ is the cross-entropy loss on 10-class role classification. The 10 classes are verb (kartā, karma, karan, etc.), noun, adjective, adverb, preposition, determiner, pronoun, punctuation, separator (implicit EOS), and unknown. Role labels are precomputed via spaCy dependency parsing on the training corpus. A RoleStreamPacker aligns these labels with the tokenizer’s windowing scheme in lockstep, ensuring 1-to-1 alignment. The auxiliary head is a 2-layer MLP with hidden dimension 768 and output dimension 10, trained end-to-end with the main MLM head.

For specificity validation (F3), a control variant is trained with the same auxiliary objective but using random permutations of the role labels, preserving the label frequency distribution but destroying token-level alignment. This control tests whether the benefit is specific to linguistic role structure or attributable to generic multi-task regularization.

F Evaluation Methodology

Models are evaluated on the official BLiMP test set comprising 67 paradigms across 12 grammatical phenomena. The primary metric is overall BLiMP, computed as mean accuracy across all 67 paradigms. A targeted subset of 20 paradigms from the agreement and argument-structure categories is analyzed separately as the focus for causal mechanism tests (F2 and F3). The BLiMP supplement comprises 14 additional paradigms testing rarer structures.

Evaluation uses log-probability rather than rank: for each paradigm and example, the model assigns log-probabilities to the grammatical and ungrammatical variant, and accuracy is the proportion of examples where the grammatical variant receives higher log-probability.

Text Average is the macro-average over BLiMP, BLiMP Supplement, EWoK (Elements of World Knowledge) [21], Entity Tracking, and COMPS (Conceptual Minimal Pair Sentences) [22] — the official BabyLM “text” track aggregate. EWoK is a cognition-inspired world-knowledge benchmark; COMPS is a minimal-pair test of property knowledge; Entity Tracking measures zero-shot in-context coreference tracking over short passages [1]. For the Strict model, Text Average is 55.99; the Strict baseline achieves approximately 54.

G Empirical Results

This section presents the supporting empirical detail behind the three findings reported in the main text, beginning with baseline comparisons and proceeding through the objective-scale, kāraka-masking, and kāraka-auxiliary analyses.

G1 Baseline Comparisons

Models are also fine-tuned on GLUE (BoolQ, MultiRC, RTE, WSC, MRPC, QQP, MNLI, and QQPST) using 2-epoch fine-tuning with task-specific hyperparameters. Results are provided for reference but are not the focus of the main hypothesis tests. The Strict-Small submission model (prabhasa-b_ss-0.1) achieves a GLUE average of 0.5807, computed as the mean of per-task best metrics. The BabyLM official baselines are Strict BLiMP 74.53 and Strict-Small BLiMP 65.08 [2].

G2 Finding 1: Objective Scale-Dependence

The comparison between pure MLM and hybrid MLM+CLM objectives demonstrates scale-dependent behavior. At 10M tokens (Strict-Small), pure MLM achieves a 3-seed mean of 63.58 ± 1.73 BLiMP (seeds 0/1/2 scoring 65.22/63.31/62.20; md5 checksums fa586488/15e33621/7a0bfa4a), while hybrid MLM+CLM achieves a 3-seed mean of 64.09 ± 0.26 BLiMP. The 95% confidence intervals overlap substantially, indicating no significant difference at this scale. However, at 100M tokens (Strict track), pure MLM achieves 73.06 BLiMP, compared to the hybrid variant at 67.57, a difference of 5.49 percentage points. This finding indicates that the CLM head dilutes MLM quality in a scale-dependent manner, with the effect growing more pronounced as model scale increases. Table G5 summarizes these results.

Table G5: Finding 1: Pure MLM vs Hybrid MLM+CLM at different scales.

Objective	Scale	Mean BLiMP	SD	95% CI / Note
Pure MLM	10M	63.58	1.73	[61.85, 65.31]
Hybrid MLM+CLM	10M	64.09	0.26	[63.83, 64.35]
Pure MLM	100M	73.06	—	Single seed
Hybrid MLM+CLM	100M	67.57	—	Single seed

G3 Finding 2: Kāraka Masking Causality

Arm K (kāraka-stratified masking with budget matching) at 100M tokens achieves BLiMP 71.77 on the full test set and 82.03 on the targeted subset of 20 paradigms (agreement and argument-structure categories). Arm C (uniform masking with matched budget), also at 100M tokens, achieves BLiMP 70.49 and 81.93 on the targeted subset (both single seed). The causal contrast on the targeted subset is $\Delta_{K-C} = +0.10$ percentage points. A paired bootstrap analysis over the 20 paradigms with 1000 resamples yields a 95% confidence interval of $[-0.99, +1.20]$, and the t -test statistic is $t < 0.1$, indicating no significant difference. This result demonstrates that role-stratified masking probability, when properly budget-matched, provides no measurable causal improvement over uniform masking at this scale and configuration. Table G6 presents this comparison.

Table G6: Finding 2: Kāraka masking (Arm K) versus uniform masking (Arm C) at 100M tokens.

Variant	Full BLiMP	Targeted Subset	Note
Arm K (Kāraka-stratified)	71.77	82.03	Single seed, budget-matched
Arm C (Uniform)	70.49	81.93	Single seed, matched budget

G4 Finding 3: Kāraka Auxiliary Objective (Five-Seed Final)

The kāraka auxiliary objective ($\lambda = 1.0$) is evaluated against a baseline ($\lambda = 0$) across five independent seeds (random states 0 through 4) at 10M tokens, with the corpus, the hybrid MLM objective, the masking, and the seed list held fixed; only the presence of the auxiliary objective varies. On the full 67-paradigm BLiMP suite, the auxiliary condition attains a 5-seed mean of 64.89 and the baseline 63.88, a difference of +1.01 percentage points. On the targeted 20-paradigm subset, the auxiliary condition attains 74.02 and the baseline 73.26. The single individually documented pair is seed 0, with auxiliary 74.97 versus baseline 72.32 on the targeted subset.

The pre-registered primary metric is the per-seed targeted-subset contrast ($\Delta_{\text{aux-base}}$). Across the five seeds the contrasts are +2.65, +0.61, +1.12, -0.22 , and -0.34 percentage points, with a 5-seed mean of +0.76 (sd 1.21, se 0.54). A paired t -test yields $t = 1.41$ (df=4) and a 95% confidence interval of $[-0.74, +2.27]$, which does not reach statistical significance. The contrast attenuates monotonically as seeds accumulate, from +2.65 at one seed to +1.46 at three seeds to +0.76 at five, indicating that the initial single-seed result reflected seed variance rather than a stable effect. The kāraka auxiliary objective therefore produces no statistically significant improvement over the baseline at five seeds and 10M token scale. Table G7 summarizes these results.

Table G7: Finding 3: Kāraka auxiliary objective (treatment, $\lambda = 1.0$) versus baseline ($\lambda = 0$) across five seeds at 10M tokens.

Condition	Full BLiMP	Targeted subset	Note
Treatment (Aux, $\lambda = 1.0$)	64.89	74.02	5-seed mean
Baseline (No Aux, $\lambda = 0$)	63.88	73.26	5-seed mean
Mean difference	+1.01	+0.76	sd 1.21, se 0.54
95% CI (targeted)	—	$[-0.74, +2.27]$	$t = 1.41$, df=4, ns

H Checkpoint Locations and Artifact Access

The Strict-Small submission model is located at qbz506/prabhasa-b_ss-0.1 on HuggingFace, trained with hybrid MLM+CLM achieving 64.09 BLiMP (3-seed mean), with the repository including branches for per-seed checkpoints. The Strict track model at qbz506/prabhasa-b_s on HuggingFace employs pure MLM and achieves 73.06 BLiMP as a single-seed run using the epoch 10 checkpoint. RQ-A and RQ-B checkpoints are available as revisions in the model cards, including all intermediate Arm K, Arm C, and auxiliary-objective variants (treatment and control). The grammar dataset is located at qbz506/prabhasa-babylm-grammar and contains morpheme inventories, role lexicons, and evaluation datasets.

I Code and Reproducibility

The main repository is `github.com/SharathSPHD/prabhasa-babylm`. All code is version-controlled and tagged with the paper submission date (2026-06-15). The primary training entry point is the configuration-driven trainer in the repository, with mechanism flags documented inline and in the per-arm configuration files under `configs/phase2/`. Evaluation is handled by `scripts/official_eval.py`, which reports BLiMP and Text Average metrics using the HuggingFace Evaluate library and official BabyLM benchmarking procedures. The analysis for F2 causal inference is in `scripts/analyze_rqA.py` (per-paradigm bootstrap), and the F3 analysis is in `scripts/analyze_rqB.py` (per-seed *t*-test). Adversarial review commentary on potential overclaims, alternative interpretations, and limitations is available in `docs/memory/tarka_rqA_rqB.md`.

I1 Locked 100M Recipe and Seed-Reproducibility

The validated 100M Strict submission (prabhasa-b_s, BLiMP 73.06) was trained at the locked code state recorded in the development history under the commit message “*LOCK pure-MLM for 100M Strict run.*” The exact recipe is: objective MLM (no CLM head), dose-arm A with zero pre-pretraining epochs, ten English epochs, RoPE, N-hot embeddings, role-stratified masking with a cosine mask schedule from 0.40 to 0.15 and frequency weight $\alpha = 0.5$, maximum sequence length 192, vocabulary 20,000, Muon optimizer at peak learning rate 1×10^{-3} , seed 0; the resulting checkpoint (best loss 0.515) is published as qbz506/prabhasa-b_s.

A reproducibility caveat is recorded here in full. Re-running the locked recipe from identical initialization under two further seeds does not recover 73.06: the additional seeds reach best masked-language-modelling losses of 1.61 and 1.82 and BLiMP scores of 58.3 and 53.6. Weight initialization is held fixed across seeds, so all trajectories begin identically and the divergence arises during optimization; a fix that re-seeded the post-initialization RNG correctly exposed genuine training stochasticity (previously, distinct -seed values had produced byte-identical checkpoints), revealing that the 100M recipe sits at the edge of an optimization instability rather than introducing a bug. The evaluation is not implicated: the submission checkpoint re-evaluates to 73.06 on the current pipeline, and an independent pseudo-log-likelihood proxy reproduces all three per-seed scores within 0.4 points. Reproducing the submitted model therefore requires the locked recipe with seed 0; making the 73-point basin reachable independent of seed is identified as future work.

J Ethical Considerations and Limitations

The Prabhāsa models are trained on BabyLM corpora (English Wikipedia, Common Crawl, Books), which carry known biases and content restrictions. The models are not intended for production deployment and are released for research purposes only. The auxiliary mechanisms incorporate Sanskrit linguistic categories; users working in low-resource language settings should verify that role assignments are appropriate for their language family before adapting the code to other languages or domains. Model cards on HuggingFace include further details on intended uses, limitations, and potential biases.

Data Availability Statement: Code: <https://github.com/SharathSPHD/prabhāsa-babylm>; models and checkpoints: https://huggingface.co/qbz506/prabhāsa-b_s, https://huggingface.co/qbz506/prabhāsa-b_ss-0.1; data: <https://huggingface.co/datasets/qbz506/prabhāsa-babylm-grammar>. **Conflicts of Interest:** The author declares no conflict of interest.

References

- [1] Alex Warstadt, Aaron Mueller, Leshem Choshen, Ethan Wilcox, Chengxu Zhuang, Juan Ciro, Rafael Mosquera, Bhargavi Paranjabe, Adina Williams, Tal Linzen, and Ryan Cotterell. Findings of the BabyLM challenge: Sample-efficient pretraining on developmentally plausible corpora. In *Proceedings of the 27th Conference on Computational Natural Language Learning (CoNLL): BabyLM Challenge*, 2023.
- [2] Jennifer Hu, Leshem Choshen, Alex Warstadt, et al. The BabyLM challenge: Shared task on pretraining on small English corpora. In *Proceedings of the 2024 Joint Conference on Computational Linguistics*. ACL, 2024.
- [3] Bimal Krishna Matilal. *The Word and the World: India's Contribution to the Study of Language*. Oxford University Press, Delhi, 1990. ISBN 9780195655124.
- [4] Paul Kiparsky and J. F. Staal. Syntactic and semantic relations in Pāṇini. *Foundations of Language*, 5(1):83–117, 1969.
- [5] George Cardona. Pāṇini's kāraṇas: Agency, animation and identity. *Journal of Indian Philosophy*, 2(3):231–306, 1974. doi: 10.1007/BF00196007.
- [6] Arun Prasad. A fast prakriyā generator for Pāṇinian Sanskrit. In *Proceedings of the 7th International Sanskrit Computational Linguistics Symposium*, 2024. ACL Anthology 2024.iscls-1.7. Accessed: 16 June 2026.
- [7] Lucas Georges Gabriel Charpentier and David Samuel. GPT or BERT: Why not both? In *Proceedings of the BabyLM Challenge at the 28th Conference on Computational Natural Language Learning (CoNLL)*, 2024. arXiv:2410.24159. Accessed: 15 June 2026.
- [8] David Samuel, Andrey Kutuzov, Lilja Øvrelid, and Erik Velldal. Trained on 100 million words and still in shape: BERT meets British National Corpus. In *Findings of the Association for Computational Linguistics: EACL 2023*, pages 1954–1974, 2023. arXiv:2303.09859. Accessed: 15 June 2026.

- [9] Lucas Georges Gabriel Charpentier and David Samuel. Not all layers are equally as important: Every layer counts BERT. In *Proceedings of the BabyLM Challenge at the 27th Conference on Computational Natural Language Learning (CoNLL)*, 2023. arXiv:2311.02265. Accessed: 15 June 2026.
- [10] Benfeng Xu, Licheng Zhang, Zhendong Mao, Quan Wang, Hongtao Xie, and Yongdong Zhang. Curriculum learning for natural language understanding. In *Proceedings of the 58th Annual Meeting of the Association for Computational Linguistics (ACL)*, pages 6095–6104, 2020.
- [11] Yonatan Belinkov, Nadir Durrani, Fahim Dalvi, Hassan Sajjad, and James Glass. What do neural machine translation models learn about morphology? In *Proceedings of the 55th Annual Meeting of the Association for Computational Linguistics (ACL)*, pages 861–872, 2017. arXiv:1704.03471. Accessed: 15 June 2026.
- [12] Jan A Botha and Phil Blunsom. Compositional Morphology for Word Representations and Language Modelling. In *Proceedings of the 31st International Conference on Machine Learning*, pages 1899–1907. PMLR, 2014.
- [13] Nāgojībhāṭṭa. *The Paribhāṣenduśekhara of Nāgojībhāṭṭa*. Government Central Book Depot, Bombay, 1868. Edited and translated with notes by F. Kielhorn; reprinted Motilal Banarsidass, ISBN 8171100465.
- [14] Jonardon Ganeri. *Semantic Powers: Meaning and the Means of Knowing in Classical Indian Philosophy*. Clarendon Press, Oxford, 1999. ISBN 9780198237884.
- [15] Amba Kulkarni et al. Designing a constraint based parser for Sanskrit. In *Sanskrit Computational Linguistics*, volume 6465 of *Lecture Notes in Computer Science*. Springer, 2010. doi: 10.1007/978-3-642-17528-2_6.
- [16] Gérard Huet. The Sanskrit Heritage Search and Analysis engine. In *Sanskrit Computational Linguistics*. Springer, 2003.
- [17] Jianlin Su, Yu Lu, Shengfeng Pan, et al. RoFormer: Enhanced transformer with Rotary Position Embedding. *arXiv preprint*, 2021. arXiv:2104.09864. Accessed: 15 June 2026.
- [18] Keller Jordan, Yuchen Jin, Vlado Boza, Jiacheng You, Franz Cesista, Laker Newhouse, and Jeremy Bernstein. Muon: An optimizer for hidden layers in neural networks. <https://kellerjordan.github.io/posts/muon/>, 2024. Blog post. Accessed: 15 June 2026.
- [19] Thomas Wolf, Lysandre Debut, Victor Sanh, Julien Chaumond, Clement Delangue, Anthony Moi, et al. Transformers: State-of-the-art natural language processing. In *Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing (EMNLP): System Demonstrations*, pages 38–45, 2020. arXiv:1910.03771. Accessed: 16 June 2026.
- [20] Alex Warstadt, Alicia Parrish, Haokun Liu, Anhad Mohananey, Wei Peng, Sheng-Fu Wang, and Samuel R. Bowman. BLiMP: The benchmark of linguistic minimal pairs for English. *Transactions of the Association for Computational Linguistics*, 8:377–392, 2020. arXiv:1912.00582. Accessed: 15 June 2026.

- [21] Anna A. Ivanova, Aalok Sathe, Benjamin Lipkin, Unnathi Kumar, Setayesh Radkani, Thomas H. Clark, Carina Kauf, Jennifer Hu, R. T. Pramod, Gabriel Grand, et al. Elements of world knowledge (EWoK): A cognition-inspired framework for evaluating basic world knowledge in language models. *Transactions of the Association for Computational Linguistics*, 2024. arXiv:2405.09605. Accessed: 16 June 2026.
- [22] Kanishka Misra, Julia Rayz, and Allyson Ettinger. COMPS: Conceptual minimal pair sentences for testing robust property knowledge and its inheritance in pre-trained language models. In *Proceedings of the 17th Conference of the European Chapter of the Association for Computational Linguistics (EACL)*, pages 2920–2941, 2023. arXiv:2210.01963. Accessed: 16 June 2026.
- [23] Alex Wang, Amanpreet Singh, Julian Michael, Felix Hill, Omer Levy, and Samuel R. Bowman. GLUE: A multi-task benchmark and analysis platform for natural language understanding. In *International Conference on Learning Representations (ICLR)*, 2019. arXiv:1804.07461. Accessed: 16 June 2026.
- [24] Matthew Honnibal, Ines Montani, Sofie Van Landeghem, and Adriane Boyd. spaCy: Industrial-strength natural language processing in Python. <https://spacy.io>, 2020.
- [25] Taku Kudo and John Richardson. SentencePiece: A simple and language independent subword tokenizer and detokenizer for neural text processing. In *Proceedings of the 2018 Conference on Empirical Methods in Natural Language Processing (EMNLP): System Demonstrations*, pages 66–71, 2018. arXiv:1808.06226. Accessed: 16 June 2026.
- [26] Rico Sennrich, Barry Haddow, and Alexandra Birch. Neural Machine Translation of Rare Words with Subword Units. *arXiv preprint*, 2016. arXiv:1508.07909. Accessed: 15 June 2026.